# Chapter 16: Sampling Plans

Algorithms for Optimization, 2nd ed.
Kochenderfer & Wheeler (MIT Press, 2026)

Lecture Notes

Based on Kochenderfer & Wheeler — Algorithms for Optimization

June 12, 2026

# Chapter 16 Overview

# Why Sampling Plans?

## The Core Problem

Evaluating a complex objective $f(\mathrm{x})$ is expensive:

- Hardware design / wing aerodynamics simulation
- Hyperparameter tuning (deep neural networks)
- Surrogate model construction

## Key Insight

Before running any optimizer, we need an initial set of design points that *covers* the search space as well as possible using a limited number of evaluations.

A **sampling plan** $X = \{\mathrm{x}^{(1)}, \ldots, \mathrm{x}^{(m)}\}$ is a set of $m$ points in the design space $\mathrm{x} \in [a_1, b_1] \times \cdots \times [a_n, b_n] \subset \mathbb{R}^n$.

## Design Space

- $n$ dimensions (variables)
- $m$ sample points
- Lower bounds $\mathrm{a} \in \mathbb{R}^n$
- Upper bounds $\mathrm{b} \in \mathbb{R}^n$

**Goal:** maximize coverage of $[\mathrm{a}, \mathrm{b}]^n$ while keeping $m$ small.

# 16.1 Full Factorial Designs

## Definition

Place $m_i$ evenly-spaced points in each dimension $i$:

$$x_{i,j} = a_i + (j-1)\frac{b_i - a_i}{m_i - 1}, \quad j = 1, \ldots, m_i$$

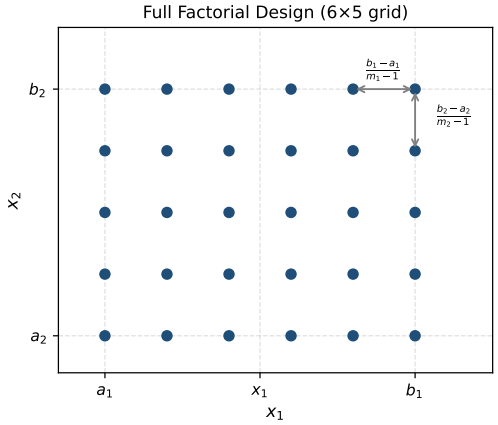
Total samples: $\prod_{i=1}^{n} m_i$

## Spacing between adjacent points

$$\Delta_i = \frac{b_i - a_i}{m_i - 1}$$

## Curse of Dimensionality

If $m_i = m$ for all $i$: total $= m^n$.
For $m = 10, n = 10$: $10^{10}$ samples!
Exponential growth makes full factorial infeasible for




Grid of $6 \times 5 = 30$ points in 2D. Spacing labelled as $\frac{b_1-a_1}{m_1-1}$ and $\frac{b_2-a_2}{m_2-1}$.

# Full Factorial — Python Implementation

## Algorithm 16.1 (Python equivalent)

Return all grid locations for a full factorial plan. `a`, `b`: lower/upper bound vectors; `m`: sample counts per dimension.

```python
import numpy as np
from itertools import product

def samples_full_factorial(a, b, m):
    """
    a : array of lower bounds (length n)
    b : array of upper bounds (length n)
    m : array of sample counts per dimension (length n)
    Returns list of all grid points as numpy arrays.
    """
    grids = [np.linspace(a[i], b[i], m[i]) for i in range(len(a))]
    return [np.array(pt) for pt in product(*grids)]

# Example: 2D, 3x3 grid on [0,1]^2
pts = samples_full_factorial([0, 0], [1, 1], [3, 3])
print(f"Total points: {len(pts)}")   # 9
print("Points:", np.round(pts, 3))
```

Listing 1: Full factorial sampling (NumPy)

# 16.2 Random Sampling

## Method

Draw $m$ points independently from the design space. For bounded variable $x_i \in [a_i, b_i]$:

$$x_i \sim \text{Uniform}(a_i, b_i)$$

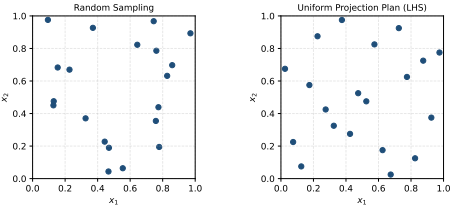
Components are *independent* across dimensions.

## Advantages

- Trivially simple to implement
- No exponential blowup with $n$
- Unbiased — no systematic structure

## Disadvantages

- **Clustering**: points tend to clump by chance
- **Gaps**: large regions can be left uncovered




Left: random sampling (note clustering).

Right: LHS — uniform projection (better coverage).

# Random Sampling — Python

```python
import numpy as np

def random_sampling(a, b, m, seed=None):
    """
    Draw m random samples from the hypercube [a, b].
    a, b : lower/upper bound arrays (length n)
    m    : number of samples
    Returns (m x n) array.
    """
    rng = np.random.default_rng(seed)
    a, b = np.asarray(a, float), np.asarray(b, float)
    return a + rng.uniform(size=(m, len(a))) * (b - a)

# Example: 20 points in [0,1]^2
X = random_sampling([0, 0], [1, 1], 20, seed=42)
print(X.shape)   # (20, 2)

# Log-uniform sampling for step-size hyperparameters
def log_uniform_sampling(a, b, m, seed=None):
    rng = np.random.default_rng(seed)
    log_a, log_b = np.log(a), np.log(b)
    return np.exp(log_a + rng.uniform(size=(m, len(a))) * (log_b - log_a))
```

Listing 2: Random sampling with NumPy

# 16.3 Uniform Projection Plans

## Definition

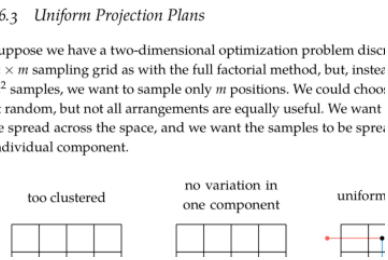
A sampling plan $X$ over an $m \times m$ grid is a **uniform projection plan** if, when projected onto any single dimension, the $m$ samples are evenly distributed (one point per row, one per column — like a Latin square).

## Latin Hypercube Sampling (LHS)

Generalization to $n$ dimensions:

1. Divide each dimension into $m$ equal strata of width $\frac{1}{m}$
2. Assign exactly one sample to each stratum in each dimension
3. Randomize within each stratum:
   $x_i^{(k)} \sim \text{Uniform}\left(\frac{\sigma_k^{(i)}-1}{m}, \frac{\sigma_k^{(i)}}{m}\right)$

where $\boldsymbol{\sigma}^{(i)}$ is a random permutation of $\{1, \ldots, m\}$


16.3 Uniform Projection Plans

Suppose we have a two-dimensional optimization problem discr
$m \times m$ sampling grid as with the full factorial method, but, instea
$m^2$ samples, we want to sample only $m$ positions. We could choos
at random, but not all arrangements are equally useful. We want
be spread across the space, and we want the samples to be spre
individual component.



Left: unconstrained random (3 in one column, none in another).
Right: uniform projection (one per column).

# Uniform Projection Plans — Python

```python
import numpy as np

def latin_hypercube_sample(m, n, seed=None):
    """
    Generate m-point Latin Hypercube Sampling in n dimensions, unit hypercube.
    m : number of samples
    n : number of dimensions
    Returns (m x n) array with values in [0, 1].
    """
    rng = np.random.default_rng(seed)
    X = np.zeros((m, n))
    for i in range(n):
        perm = rng.permutation(m)          # random permutation of strata
        offsets = rng.uniform(size=m)      # uniform within each stratum
        X[:, i] = (perm + offsets) / m
    return X

# Scale to [a, b]
def lhc_sample(a, b, m, seed=None):
    a, b = np.asarray(a, float), np.asarray(b, float)
    X01 = latin_hypercube_sample(m, len(a), seed)
    return a + X01 * (b - a)

# Example
X = lhc_sample([0, 0], [1, 1], 10, seed=7)
print("LHS plan (10 pts, 2D):\n", np.round(X, 3))
```

# Uniform Projection Plans — Properties

## Key Property

For any one-dimensional projection onto dimension $i$, the $m$ points each fall in a distinct stratum $\left[\frac{k-1}{m}, \frac{k}{m}\right]$ for $k = 1, \ldots, m$.

## Numerical Example ($m = 5$, $n = 2$)

Strata in $x_1$:
$[0, 0.2), [0.2, 0.4), [0.4, 0.6), [0.6, 0.8), [0.8, 1]$
Each stratum gets *exactly one* point.
Same guarantee for $x_2$.

## Limitation

Uniform projection is guaranteed only for 1D marginals. Pairs of dimensions may still cluster (in higher $n$).

## Algorithm 16.2 (Python)

`sample_uniform_projection(m)` generates an $m$-point uniform projection plan over $\{1, \ldots, m\}^n$:

1. For each dimension $i$: draw random permutation $\sigma^{(i)}$ of $1 \ldots m$
2. Return $\{(\sigma_k^{(1)}, \sigma_k^{(2)}, \ldots, \sigma_k^{(n)})\}_{k=1}^{m}$

Total possible plans in 2D: $m!$ (only $m$ of $m^m$ grids are valid UPPs).
For $m = 10$: $10! = 3{,}628{,}800$ possible plans.

# 16.4 Stratified Sampling

## Concept

Divide the design space into non-overlapping **strata** (sub-regions). Draw at least one sample from each stratum.

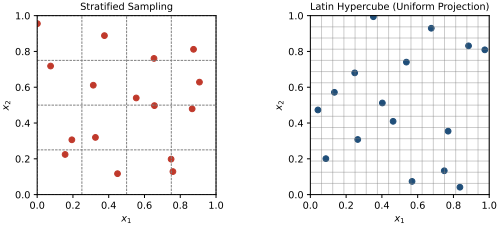
$$X = X_1 \cup X_2 \cup \cdots \cup X_k, \quad X_i \cap X_j = \emptyset$$

This prevents large empty regions from occurring.

## Relation to Projection Plans

Stratified sampling $\supseteq$ LHS:

- **Pure stratified**: one point per 2D cell (factorial structure, $m^2$ total)
- **LHS / UPP**: one point per row *and* per column, $m$ total
- LHS is a more efficient form of stratified sampling




Left: grid-stratified (one point per 2D cell).

Right: Latin Hypercube (one per row & column stratum).

# Stratified Sampling — Python

```python
import numpy as np

def stratified_sample(a, b, m_per_dim, seed=None):
    """
    Grid-stratified sampling: one random point per cell.
    a, b         : bound vectors (length n)
    m_per_dim    : number of strata per dimension (list of ints)
    Returns array of shape (prod(m_per_dim), n).
    """
    from itertools import product
    rng = np.random.default_rng(seed)
    a, b = np.asarray(a, float), np.asarray(b, float)
    n = len(a)
    # stratum edges for each dimension
    edges = [np.linspace(a[i], b[i], m_per_dim[i] + 1) for i in range(n)]
    pts = []
    for idx in product(*[range(m) for m in m_per_dim]):
        pt = [rng.uniform(edges[i][idx[i]], edges[i][idx[i]+1])
              for i in range(n)]
        pts.append(pt)
    return np.array(pts)

# Example: 3x3 strata in 2D
X = stratified_sample([0,0], [1,1], [3,3], seed=0)
print(f"Points: {len(X)}")  # 9
```

Listing 4: Stratified sampling in Python

# 16.5 Space-Filling Metrics — Overview

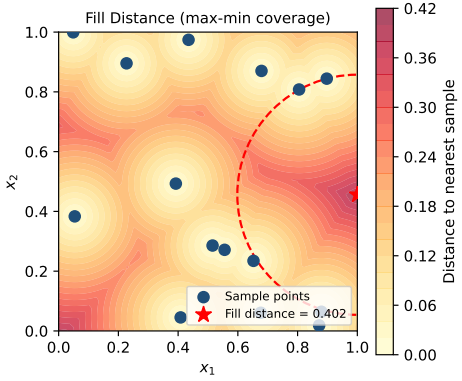
To *compare* sampling plans, we need a quantitative metric. Two key families:

1. **Discrepancy** — how much does the empirical distribution deviate from uniform?
2. **Fill Distance** — what is the largest uncovered region?

Both measure different aspects of "how well does $X$ cover $\mathcal{X}$?"

## Setup

Design space: $\mathcal{X} = [0,1]^n$ (unit hypercube)
Sampling plan: $X = \{\mathrm{x}^{(1)}, \ldots, \mathrm{x}^{(m)}\}$




Heatmap of distance-to-nearest-sample. Fill distance = radius of largest empty circle (red star).

## Definition (Star Discrepancy)

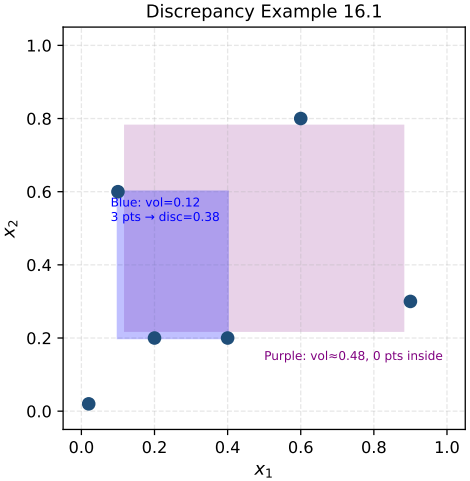
For a set $X$ of $m$ points and a rectangular subset $H \subseteq [0,1]^n$:

$$D(X) = \sup_{H \subseteq [0,1]^n} \left| \frac{\#(X \cap H)}{m} - \mathrm{Vol}(H) \right|$$

where the supremum is over all axis-aligned rectangles $H$.

## Interpretation

- $\frac{\#(X \cap H)}{m}$: fraction of points in $H$
- $\mathrm{Vol}(H)$: fraction of volume in $H$
- $D(X)$ = maximum mismatch between empirical and uniform
- **Low discrepancy** $\Rightarrow$ better coverage




**Example 16.1:** Blue rectangle: $x_1 \in [\frac{1}{10}, \frac{2}{5}]$, $x_2 \in [\frac{1}{5}, \frac{3}{5}]$, Vol $= 0.12$, contains 3 pts. Discrepancy: $|\frac{3}{5} - 0.12| = 0.38$

# 16.5.1 Discrepancy — Worked Example I

# 16.5.1 Discrepancy — Worked Example II

## Example 16.1 from the book

Consider the 6-point set:

$$X = \left\{ \begin{bmatrix} 1/5 \\ 1/5 \end{bmatrix}, \begin{bmatrix} 2/5 \\ 1/5 \end{bmatrix}, \begin{bmatrix} 1/10 \\ 3/5 \end{bmatrix}, \begin{bmatrix} 9/10 \\ 3/10 \end{bmatrix}, \begin{bmatrix} 1/50 \\ 1/50 \end{bmatrix}, \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} \right\}$$

**Blue rectangle**: $x_1 \in [1/10, 2/5]$, $x_2 \in [1/5, 3/5]$

- Volume $= (2/5 - 1/10) \times (3/5 - 1/5) = 0.3 \times 0.4 = 0.12$
- Contains points: $(1/5, 1/5)$, $(2/5, 1/5)$, $(1/10, 3/5)$ — **3 points**
- Discrepancy measure: $|3/6 - 0.12| = |0.5 - 0.12| = 0.38$

**Purple rectangle**: $x_1 \in [1/10 + \epsilon, 9/10 - \epsilon]$, $x_2 \in [1/5 + \epsilon, 4/5 - \epsilon]$

- As $\epsilon \to 0$: Volume $\to (0.8) \times (0.6) = 0.48$
- Contains **0 points** (just misses all of them)
- Discrepancy: $|0/6 - 0.48| = 0.48$ (even worse!)

The **supremum** (over all rectangles) gives $D(X) = 0.48$. Note: we need the *supremum* because the maximum is approached in the limit $\epsilon \to 0$.

# 16.5.2 Fill Distance (Packing Distance)

## Fill Distance

The **fill distance** (covering radius) measures the worst-case distance from any point in $\mathcal{X}$ to its nearest sample:

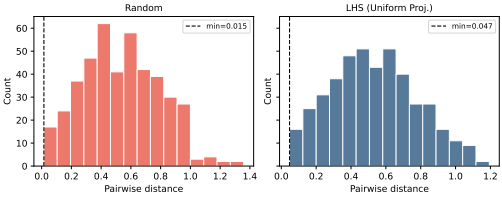
$$d_{\text{fill}}(X, \mathcal{X}) = \sup_{z \in \mathcal{X}} \min_{x \in X} \|z - x\|_p$$

## Pairwise Distances (Algorithm 16.3)

To assess coverage, compute all pairwise $L_p$ distances:

$$D_{\text{pairs}}(X, p) = \left\{ \|x^{(i)} - x^{(j)}\|_p \;\middle|\; i < j \right\}$$

Minimum pairwise distance = *packing distance*.
Want: large min distance AND small fill distance.




Pairwise distance histograms for 30-point plans. LHS (right) has larger minimum pairwise distances.

# Pairwise Distances — Python

```python
import numpy as np
from scipy.spatial.distance import pdist

def pairwise_distances(X, p=2):
    """
    X : (m x n) array of sample points
    p : L_p norm (default: Euclidean p=2)
    Returns array of all m*(m-1)/2 pairwise distances.
    """
    # Direct loop (as in book)
    m = len(X)
    dists = [np.linalg.norm(X[i] - X[j], ord=p)
             for i in range(m-1) for j in range(i+1, m)]
    return np.array(dists)

# Using scipy (faster):
def pairwise_distances_scipy(X, p=2):
    return pdist(X, metric='minkowski', p=p)

# Compare two plans
X_rand = np.random.default_rng(0).uniform(0, 1, (20, 2))
X_lhc  = latin_hypercube_sample(20, 2, seed=0)  # from earlier

dr = pairwise_distances(X_rand)
dl = pairwise_distances(X_lhc)
print(f"Random: min dist = {dr.min():.4f}")
print(f"LHS:    min dist = {dl.min():.4f}")  # typically larger
```

# 16.5.3 Morris-Mitchell Criterion

## Morris-Mitchell $\Phi_q$ Criterion (Eq. 16.2)

For a sampling plan $X$ with pairwise distances $\{d_i\}$:

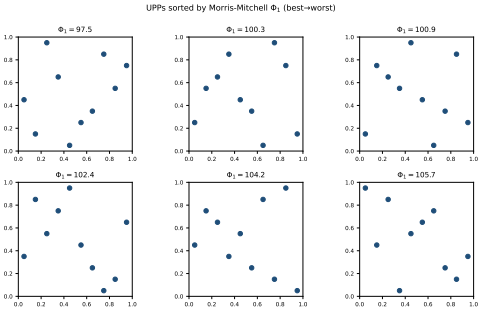
$$\Phi_q(X) = \left( \sum_i d_i^{-q} \right)^{1/q}$$

where $q > 0$ is a tunable parameter.

- $q$ large: emphasizes the *smallest* distance
- $q = 1$: harmonic mean-like; spread over all distances
- We want to **minimize** $\Phi_q$

## Optimization Problem (Eq. 16.3)

$$\underset{X}{\text{minimize}} \; \underset{\sigma}{\text{maximize}} \; \Phi_q(X)$$




Six random LHS plans sorted by $\Phi_1$. Lower $\Phi_1$ = better space-filling.

# Morris-Mitchell — Python + Mutation Algorithm

```python
import numpy as np

def phi_q(X, q=1):
    """Morris-Mitchell criterion. Minimize this."""
    dists = pdist(X)          # all pairwise L2 distances
    return np.sum(dists**(-q))**(1/q)

def compare_sampling_plans(A, B, p=2, q=1):
    """Return 1 if A is more space-filling than B, -1 if B, 0 if equal."""
    da = np.sort(pdist(A, metric='minkowski', p=p))
    db = np.sort(pdist(B, metric='minkowski', p=p))
    for a, b in zip(da, db):
        if a < b: return -1   # A has smaller min distance => B better
        if a > b: return  1
    return 0

def mutate(X):
    """Swap two values in a random column (Algorithm 16.5 in book).
    Maintains uniform projection property."""
    X2 = X.copy()
    m, n = X2.shape
    col = np.random.randint(n)
    i, j = np.random.choice(m, 2, replace=False)
    X2[i, col], X2[j, col] = X2[j, col], X2[i, col]
    return X2

# Stochastic optimization of LHS
```

# 16.6 Space-Filling Subsets

## Problem

Given a large set $X$ of design points, find a *subset* $S \subset X$, $|S| = m$, that still "maximally fills" $X$.

$$d(A, B) = \max_{x \in A} \min_{y \in B} \|x - y\|_p$$

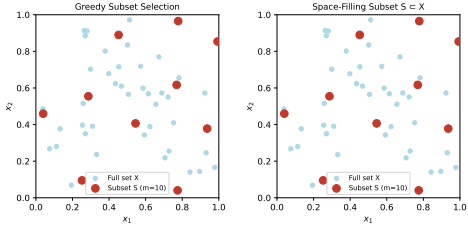
Minimize the fill distance: $\min_S \; d(X, S)$.

## Application

Multifidelity models: use $X$ (cheap simulation) to identify $m$ design points to evaluate with expensive high-fidelity model. E.g., wind-tunnel testing of aircraft wing designs.

## Optimization

Exact minimization is NP-hard. Two practical approaches: (1) Greedy selection (Alg. 16.8), (2)




Blue dots: full candidate set $X$. Red dots: space-filling subset $S \subset X$ (greedy selection, $m = 10$).

# 16.6 Greedy Subset Selection (Algorithm 16.8)

```python
import numpy as np
from scipy.spatial.distance import cdist

def greedy_subset_selection(X, m, p=2):
    """
    Algorithm 16.8: Greedy subset selection.
    Builds an m-point subset that minimizes fill distance d(X, S).
    X : (N x n) array --- full candidate set
    m : desired subset size
    p : L_p norm
    Returns list of indices into X.
    """
    N = len(X)
    rng = np.random.default_rng(0)
    S = [rng.integers(N)]            # start with one random point

    for _ in range(m - 1):
        # For each candidate NOT in S:
        # find its distance to the current subset S
        S_arr = X[S]
        # cdist: shape (N, |S|)
        dists = cdist(X, S_arr, metric='minkowski', p=p)
        # distance from each x to nearest point in S
        d_to_S = dists.min(axis=1)
        # mask out already-selected points
        d_to_S[S] = -np.inf
        # pick the point FARTHEST from S  (minimizes fill distance)
```

# 16.6 Exchange Subset Selection (Algorithm 16.9)

```
def exchange_subset_selection(X, m, p=2):
    """
    Algorithm 16.9: Exchange-based subset selection.
    Iteratively swaps elements of S with points in X \ S
    to reduce fill distance.
    """
    from scipy.spatial.distance import cdist
    rng = np.random.default_rng(0)
    S_idx = list(rng.choice(len(X), m, replace=False))

    def fill_dist(S_idx):
        S = X[S_idx]
        dists = cdist(X, S).min(axis=1)
        return dists.max()

    delta = fill_dist(S_idx)
    done = False
    while not done:
        best_pair = None
        for i in range(m):
            old = S_idx[i]
            for j, x in enumerate(X):
                if j in S_idx:
                    continue
                S_idx[i] = j
                delta_prime = fill_dist(S_idx)
                if delta_prime < delta:
```

# 16.7 Quasi-Random Sequences

## Definition

**Quasi-random sequences** (low-discrepancy sequences) are *deterministic* sequences that fill the unit hypercube more uniformly than pseudo-random sequences.

Key property: when $\mathrm{x}^{(k)}$ is sampled, the error in Monte Carlo integration satisfies:

$$\left| \frac{1}{m} \sum_{k=1}^{m} f(\mathrm{x}^{(k)}) - \int f \right| = \mathcal{O}\left( \frac{(\log m)^n}{m} \right)$$

versus $\mathcal{O}(m^{-1/2})$ for random sampling.

## Note

These sequences are *not* random. They are called



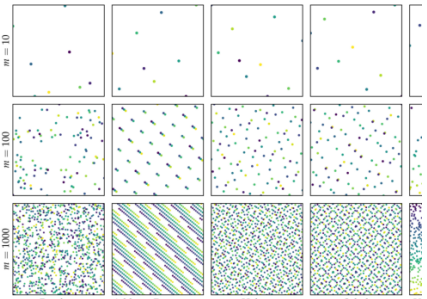



Figure 16
pling pla
Samples a
the order
pled. The
was gener

# 16.7.1 Additive Recurrence

## Method (Eq. 16.7)

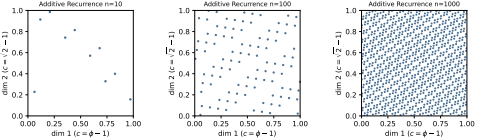
Generate a sequence in $[0, 1]$ using:

$$x^{(k+1)} = \left(x^{(k)} + c\right) \bmod 1$$

where $c$ is **irrational**. Explicit form: $x^{(k)} = (x^{(0)} + kc) \bmod 1$.

## Choice of $c$

- Best 1D: $c = \phi - 1 = \frac{\sqrt{5}-1}{2} \approx 0.618$ (golden ratio conjugate)
- For $n$ dimensions: use $c_i = \sqrt{p_i}$ where $p_i$ is the $i$-th prime
- **Must be irrational**: rational $c = a/b$ gives period $b$ (sequence repeats!)




Additive recurrence at $n = 10, 100, 1000$ samples. Diagonal stripes are visible — correlation between dimensions.

# Additive Recurrence — Python (Algorithm 16.10)

```python
import numpy as np

def filling_set_additive_recurrence(m, c=None):
    """
    1D additive recurrence sequence.
    m : number of points
    c : step size (default: golden ratio conjugate phi-1)
    """
    if c is None:
        phi = (1 + np.sqrt(5)) / 2
        c = phi - 1    # ~0.6180339887...
    x0 = np.random.rand()
    return np.array([(x0 + k * c) % 1.0 for k in range(1, m + 1)])

def filling_set_additive_recurrence_nd(m, n):
    """
    n-dimensional additive recurrence.
    Uses c_i = sqrt(prime_i) for dimension i.
    """
    def nth_prime(k):
        """Return the k-th prime (1-indexed)."""
        primes, x = [], 2
        while len(primes) < k:
            if all(x % p != 0 for p in primes):
                primes.append(x)
            x += 1
        return primes[k-1]
```

# 16.7.2 Halton Sequence

## Van der Corput Construction

For base $b$, the $i$-th Halton value is obtained by *reflecting* the base-$b$ representation of $i$ around the decimal point:

$$i = \sum_{k=0}^{\infty} a_k(i)\, b^k \quad \Rightarrow \quad \text{halton}(i, b) = \sum_{k=0}^{\infty} a_k(i)\, b^{-(k+1)}$$

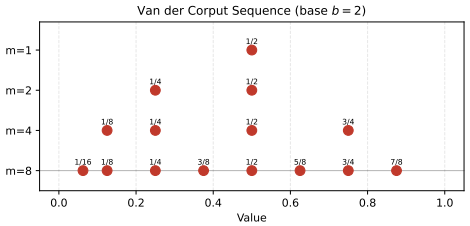
## Examples (Eqs. 16.10–16.11)

Base $b = 2$:

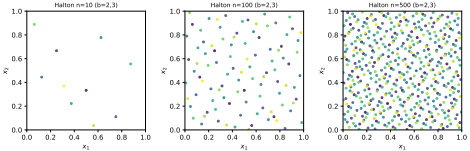
$$X = \left\{ \frac{1}{2}, \frac{1}{4}, \frac{3}{4}, \frac{1}{8}, \frac{3}{8}, \frac{5}{8}, \frac{7}{8}, \frac{1}{16}, \dots \right\}$$

Base $b = 5$:

$$X = \left\{ \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, \frac{1}{25}, \frac{6}{25}, \frac{11}{25}, \dots \right\}$$




Van der Corput sequence (base 2) progressively fills $[0, 1]$. Each new point fills the largest gap.




2D Halton (bases 2 and 3) at $n = 10, 100, 500$.

# Halton Sequence — Python (Algorithm 16.11)

```python
import numpy as np

def halton(i, b):
    """
    i-th element of the van der Corput sequence in base b.
    i : 1-indexed integer
    b : base (must be prime for coprimality)
    """
    result, f = 0.0, 1.0
    while i > 0:
        f /= b
        result += f * (i % b)
        i = i // b
    return result

def filling_set_halton(m, b=2):
    """1D Halton sequence of length m in base b."""
    return np.array([halton(i, b) for i in range(1, m + 1)])

def filling_set_halton_nd(m, n):
    """n-dimensional Halton sequence using first n prime bases."""
    from sympy import prime   # or use a precomputed list
    primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]  # first 10 primes
    assert n <= len(primes), f"Need at most {len(primes)} dimensions"
    seqs = [filling_set_halton(m, b=primes[i]) for i in range(n)]
    return np.column_stack(seqs)
```

# 16.7.3 Sobol Sequence

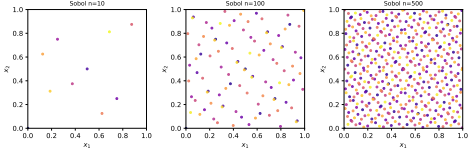
## Sobol Sequences

Sobol sequences are $n$-dimensional quasi-random sequences based on **base-2** digit scrambling using direction numbers.

- Each dimension uses a different set of *direction numbers* $v_1, v_2, \ldots$ (derived from primitive polynomials over $\mathbb{F}_2$)
- The $k$-th sample is: $x_i^{(k)} = k_1 v_1 \oplus k_2 v_2 \oplus \cdots$ (bitwise XOR)
- Sobol sequences are better than Halton for higher dimensions (Halton shows correlation for large prime bases)

## Python: scipy.stats.qmc

SciPy provides a production-quality Sobol generator.




Sobol sequence in 2D at $n = 10, 100, 500$.

Points fill space very uniformly (diagonal structure visible at small $n$).

## Important

Best results when $m$ is a power of 2 (Sobol has exact equidistribution at $2^k$ points).

# Sobol Sequence — Python

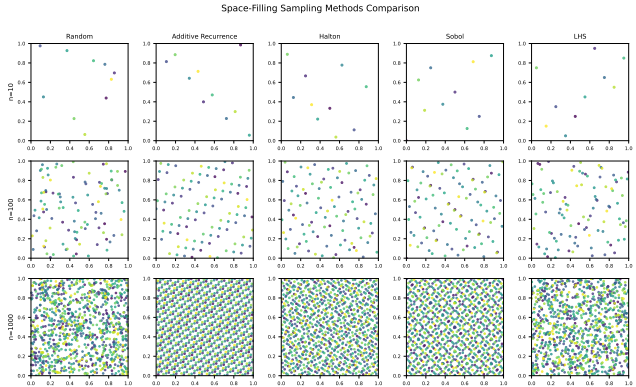
```python
import numpy as np
from scipy.stats import qmc

# --- Sobol sequence ------------------------------------------------
def sobol_sample(m, n, scramble=True, seed=42):
    """
    Generate m Sobol points in n dimensions.
    scramble: use randomized (Owen) scrambling for better uniformity
    For best equidistribution, choose m = 2^k.
    """
    sampler = qmc.Sobol(d=n, scramble=scramble, seed=seed)
    X = sampler.random_base2(m=int(np.log2(m)))  # m must be power of 2
    return X

# Scale to [a, b]
def sobol_sample_scaled(a, b, m, scramble=True, seed=42):
    a, b = np.asarray(a, float), np.asarray(b, float)
    X01 = sobol_sample(m, len(a), scramble, seed)
    return qmc.scale(X01, a, b)

# --- Halton sequence (scipy) ---------------------------------------
def halton_sample(a, b, m, seed=42):
    a, b = np.asarray(a, float), np.asarray(b, float)
    sampler = qmc.Halton(d=len(a), seed=seed)
    X01 = sampler.random(n=m)
    return qmc.scale(X01, a, b)
```

# Comparison of All Methods




Rows: $n = 10, 100, 1000$ points. Columns: Random, Additive Recurrence, Halton, Sobol, LHS (Uniform Projection). Colors encode order of sampling.

## Observations

- **Random**: clusters and gaps
- **Additive Recurrence**: diagonal banding
- **Halton**: excellent 2D coverage, may show patterns in high-$n$
- **Sobol**: very uniform, best for moderate $n$
- **LHS**: one-per-stratum guarantee, but no inter-dimensional structure

## Rule of Thumb

For $n \leq 10$ dims: Sobol / Halton.
For any $n$: optimized LHS (Morris-Mitchell).
For subsets: greedy or exchange selection

# Summary I

## Chapter 16 Key Concepts

1. **Full Factorial**: evaluates all combinations on a grid. $\mathcal{O}(m^n)$ samples — only practical for small $n$.
2. **Random Sampling**: draws $m$ independent uniform samples. Simple but prone to clustering.
3. **Uniform Projection Plans (LHS)**: one point per stratum in each dimension. Better 1D marginals; $\mathcal{O}(m)$ samples.
4. **Stratified Sampling**: generalization that ensures sub-region coverage; LHS is a special case.
5. **Discrepancy**: measures deviation of empirical distribution from uniform over all axis-aligned rectangles $H$.
6. **Fill Distance**: largest radius uncovered ball in $\mathcal{X}$.
7. **Morris-Mitchell Criterion** $\Phi_q$: minimizes $(\sum d_i^{-q})^{1/q}$; encourages large minimum pairwise distance.

# Summary II

## Key Algorithms

| Algorithm | Method | Use Case |
|---|---|---|
| 16.1 | Full factorial grid | Small $n$ (grid search) |
| 16.2 | Uniform projection plan | LHS generation |
| 16.3 | Pairwise distances | Plan evaluation |
| 16.4 | Compare plans | Ranking plans |
| 16.5 | Mutation | LHS optimization |
| 16.6 | Morris-Mitchell $\Phi_q$ | Quality metric |
| 16.7 | Max-fill distance | Subset metric |
| 16.8 | Greedy subset | Space-filling subset |
| 16.9 | Exchange subset | Improved subset |
| 16.10 | Additive recurrence | Quasi-random |
| 16.11 | Halton | Quasi-random |

# Summary III

## Monte Carlo Integration Rate

| Method | Error rate |
|---|---|
| Random (MC) | $\mathcal{O}(m^{-1/2})$ |
| Quasi-random (QMC) | $\mathcal{O}((\log m)^n/m)$ |

For large $m$ and small $n$, QMC is much faster to converge.

## Python Libraries

- `scipy.stats.qmc`: Sobol, Halton, LHS
- `numpy.random`: pseudo-random
- `pyDOE2`: Latin Hypercube (extended)
- `SALib`: sensitivity analysis designs

# 16.9 Selected Exercises I

## Exercise 16.1 — Curse of Dimensionality

An $n$-dimensional hypercube of side $\ell$ filling half the volume of the unit hypercube satisfies $\ell^n = 1/2$, so:

$$\ell = 2^{-1/n} \xrightarrow{n\to\infty} 1$$

For $n = 2$: $\ell = 1/\sqrt{2} \approx 0.707$. For $n = 10$: $\ell = 2^{-0.1} \approx 0.933$. The side length approaches 1, meaning we need to search almost the entire space to cover half its volume!

## Exercise 16.2 — Concentration of Measure

For a random point x uniform in the $n$-ball:

$$P(\|\mathrm{x}\|_2 > 1-\epsilon) = 1 - P(\|\mathrm{x}\|_2 \le 1-\epsilon) = 1-(1-\epsilon)^n \to 1$$

As $n \to \infty$, *all* mass concentrates near the surface. Random sampling becomes increasingly inefficient in high dimensions.

# 16.9 Selected Exercises II

## Exercise 16.3 — Morris-Mitchell & Translation Invariance

Sampling plan $X = \{[\cos(2\pi i/10), \sin(2\pi i/10)]\}_{i=1}^{10}$. Adding $[2, 3]$ to every point: $\Phi_2(X)$ **does not change**.

*Reason*: $\Phi_q$ depends only on *pairwise distances*, and translation shifts all points equally, leaving all pairwise distances $\|\mathrm{x}^{(i)} - \mathrm{x}^{(j)}\|$ unchanged.

## Exercise 16.4 — Why $c$ Must Be Irrational

If $c = a/b$ (rational), the additive recurrence:

$$x^{(k)} = x^{(0)} + k\frac{a}{b} \pmod 1$$

After $b$ steps: $x^{(k+b)} = x^{(0)} + (k+b)\frac{a}{b} \pmod 1 = x^{(k)} + a \pmod 1 = x^{(k)}$. The sequence is *periodic* with period $b$ and only covers $b$ distinct points, not a dense filling of $[0, 1]$.

# Chapter 16 — Take-Home Messages

## Key Principles

- Function evaluations are expensive: choose sample points wisely
- Full factorial is impractical for $n > 4$
- Latin Hypercube (UPP) gives good 1D coverage with only $m$ points
- Quasi-random sequences (Halton, Sobol) beat random for integration
- Optimize LHS with Morris-Mitchell criterion $\Phi_q$
- For subset selection: greedy or exchange algorithms

## Design Space Coverage

As dimension $n$ grows:

- Volume concentrates in corners
- All points near the surface of high-dim balls
- More samples needed to cover the same fraction
- No sampling plan fully solves the curse of dimensionality

*"Covering the space of designs is the first step toward finding the best design."*